MP2

Single core to calculate pi

pi estimate: 3.142067 (error: 0.000474)
Serial time: 0.975s

Parallel

1 workers: 0.690s pi=3.140840
2 workers: 0.353s pi=3.141587
3 workers: 0.253s pi=3.141878
4 workers: 0.212s pi=3.142228
5 workers: 0.204s pi=3.141187
6 workers: 0.191s pi=3.141656
7 workers: 0.193s pi=3.141954
8 workers: 0.188s pi=3.142312
9 workers: 0.180s pi=3.142524
10 workers: 0.167s pi=3.141581

Milestone 1

The result

Arrays equal: True
Output of refactored mandlebrot serial function matches exactly with output of original Numba mandelbrot function from mandelbrot_function.py
Both functions produce the same Mandelbrot set data (a 512x512 array of iteration counts).
Every pixel in the grid has the identical value in both versions.

Milestone 2

Verification Results:
Serial vs L03: True (matches the reference Numba implementation).
Parallel vs Serial: True (parallel output is identical to serial).

Proves scalability : can use multiple CPU cores for faster computation
Efficiency : Low IPC overhead by computing coordinates on-the-fly in each worker.
Correctness: Verified to produce the same results as the serial version.

Milestone 3

Serial: 0.008s
1 workers: 0.009s speedup=0.90 efficiency=0.90
2 workers: 0.005s speedup=1.66 efficiency=0.83
3 workers: 0.007s speedup=1.19 efficiency=0.40
4 workers: 0.005s speedup=1.79 efficiency=0.45

```
 5 workers: 0.005s speedup=1.62 efficiency=0.32
 6 workers: 0.004s speedup=2.15 efficiency=0.36
 7 workers: 0.004s speedup=2.05 efficiency=0.29
 8 workers: 0.004s speedup=2.38 efficiency=0.30
 9 workers: 0.003s speedup=2.48 efficiency=0.28
10 workers: 0.003s speedup=2.48 efficiency=0.25
```

Analyse of computation scale with increased number of worker process

Parallel Mandelbrot — MP2 M1
Chunked Mandelbrot

Serial: 0.008s

```
--- Verification: n_chunks = n_workers ---
1 workers: Output matches serial = True
2 workers: Output matches serial = True
3 workers: Output matches serial = True
4 workers: Output matches serial = True

--- Benchmark: n_chunks = n_workers ---
 1 workers: 0.010s speedup=0.88 efficiency=0.88
 2 workers: 0.005s speedup=1.69 efficiency=0.85
 3 workers: 0.007s speedup=1.19 efficiency=0.40
 4 workers: 0.005s speedup=1.73 efficiency=0.43
 5 workers: 0.005s speedup=1.55 efficiency=0.31
 6 workers: 0.004s speedup=2.08 efficiency=0.35
 7 workers: 0.004s speedup=2.02 efficiency=0.29
 8 workers: 0.004s speedup=2.38 efficiency=0.30
 9 workers: 0.004s speedup=2.17 efficiency=0.24
10 workers: 0.003s speedup=2.60 efficiency=0.26
```

2.6x speedup at 10 workers (26% efficiency)

```
--- Benchmark: n_chunks = 4 × n_workers ---
 1 workers,  4 chunks: 0.010s speedup=0.87 efficiency=0.87
 2 workers,  8 chunks: 0.005s speedup=1.62 efficiency=0.81
 3 workers, 12 chunks: 0.004s speedup=2.26 efficiency=0.75
 4 workers, 16 chunks: 0.003s speedup=2.76 efficiency=0.69
 5 workers, 20 chunks: 0.003s speedup=3.11 efficiency=0.62
 6 workers, 24 chunks: 0.003s speedup=3.12 efficiency=0.52
 7 workers, 28 chunks: 0.003s speedup=3.28 efficiency=0.47
```

8 workers, 32 chunks: 0.002s speedup=3.48 efficiency=0.44
9 workers, 36 chunks: 0.002s speedup=3.45 efficiency=0.38
10 workers, 40 chunks: 0.003s speedup=3.08 efficiency=0.31

3.1x speedup at 10 workers (31% efficiency)

All parallel implementation match serial output perfectly
Load balancing improvement : with 4x chunks per worker we can get a significantly better speedup
n_chunks = n_workers - 2.6x speedup at 10 workers (26% efficiency)

n_chunks = 4×n_workers - 3.1x speedup at 10 workers (31% efficiency)

This shows that small chunks improve work distribution across the workers , especially making sense since Mandelbrot has variable work per row

Optimal Chunk Size

milestone 2

Chunk decoupling results (n_workers = 10, L04 optimum):

n chunks	time (s)	vs. 1×	LIF
10	0.003	2.46	3.062
20	0.002	3.56	1.807
40	0.003	3.05	2.284
80	0.003	3.35	1.985
160	0.003	3.36	1.972

Optimal configuration: **20 chunks** (2× n_workers) with LIF = 1.807

Since lower LIF(measure load imbalance) better load balancing . optimal is 2x chunks per worker

Comprehensive analysis

Serial: 0.035s
8 chunks 0.013s 2.8x LIF=1.86
16 chunks 0.010s 3.4x LIF=1.36

32 chunks 0.009s 4.0x LIF=0.98
64 chunks 0.009s 4.0x LIF=0.99
128 chunks 0.009s 4.0x LIF=0.99

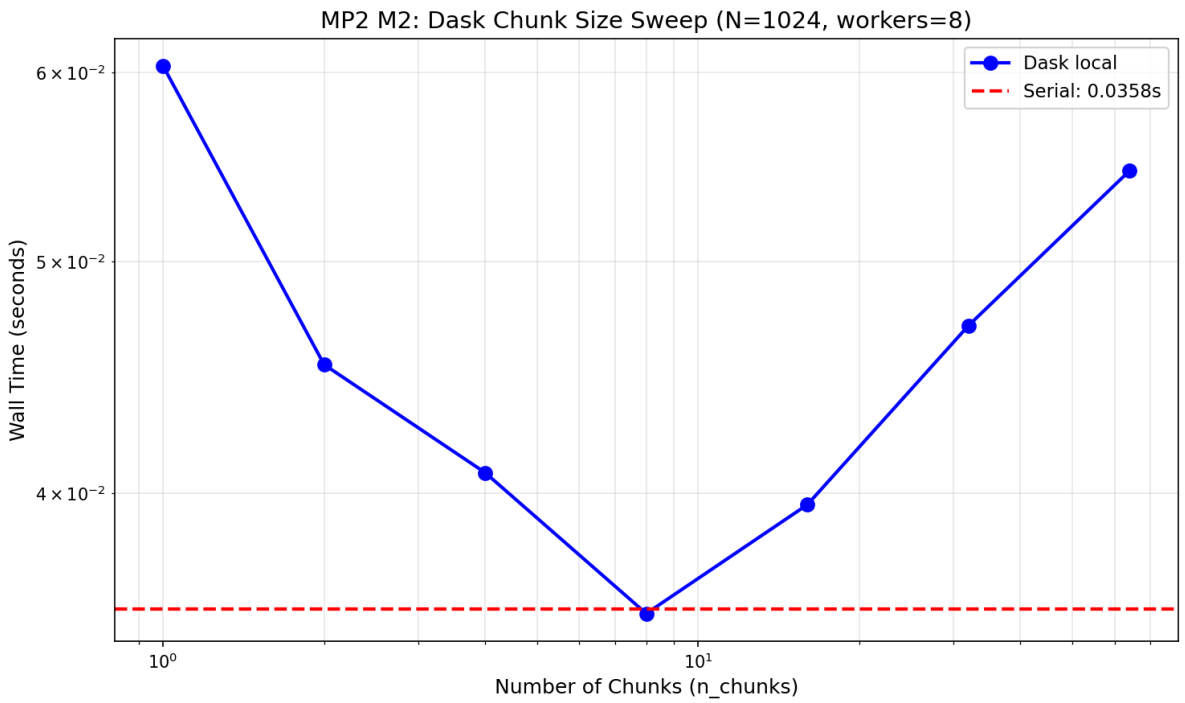
Serial: 0.035s is baseline single threaded total time
8 chunks gives 2.8 x speedup , but high LIF = 1.86 => imbalance overhead still high
16 chunks is better 3.4 x speedup , LIF = 1.36 => less imbalance
32 chunks is best 4x speedup , LIF = 0.98 very minimal
Rest same no additional benefits

MP2 M1 33 Dask Mandelbrot — Local

Implemented local dask version of a parallel mandelbrot.
Same computation as serial, but split into task and run in parallel

Dask dashboard: http://127.0.0.1:8787/status
Warm up serial and Dask JIT...
Correctness np.array_equal(serial, dask) = True
Dask local (n_workers=8, n_chunks=8): median time = 0.037s

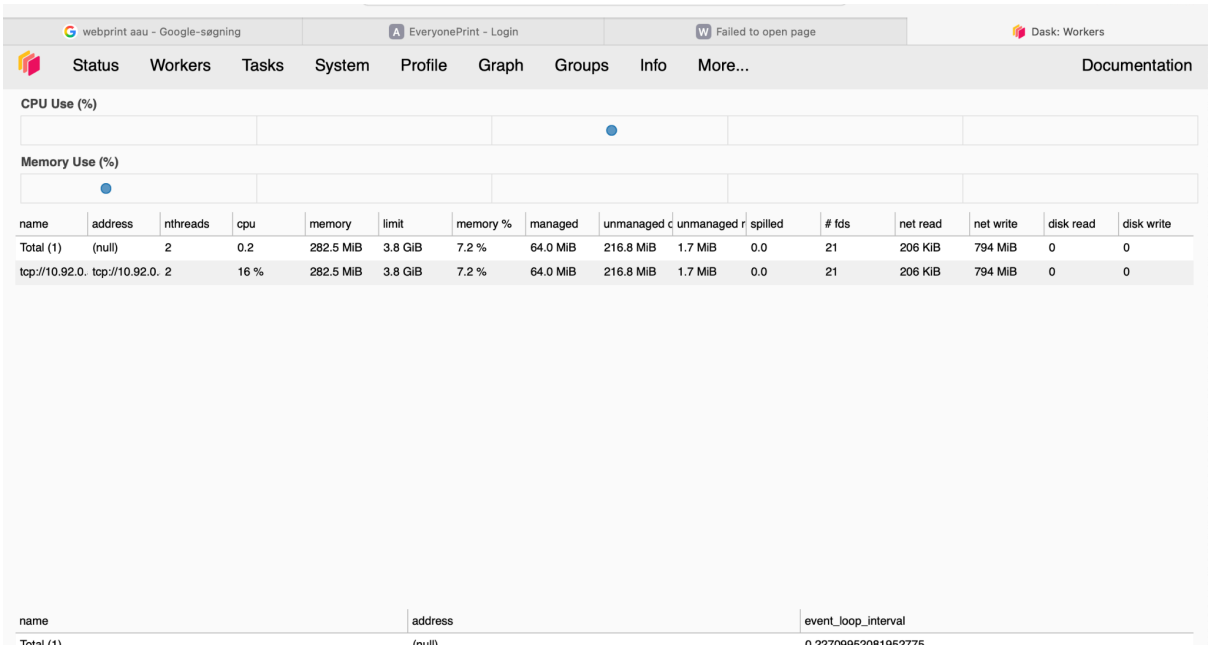
MP2 M2: Chunk Size Sweep

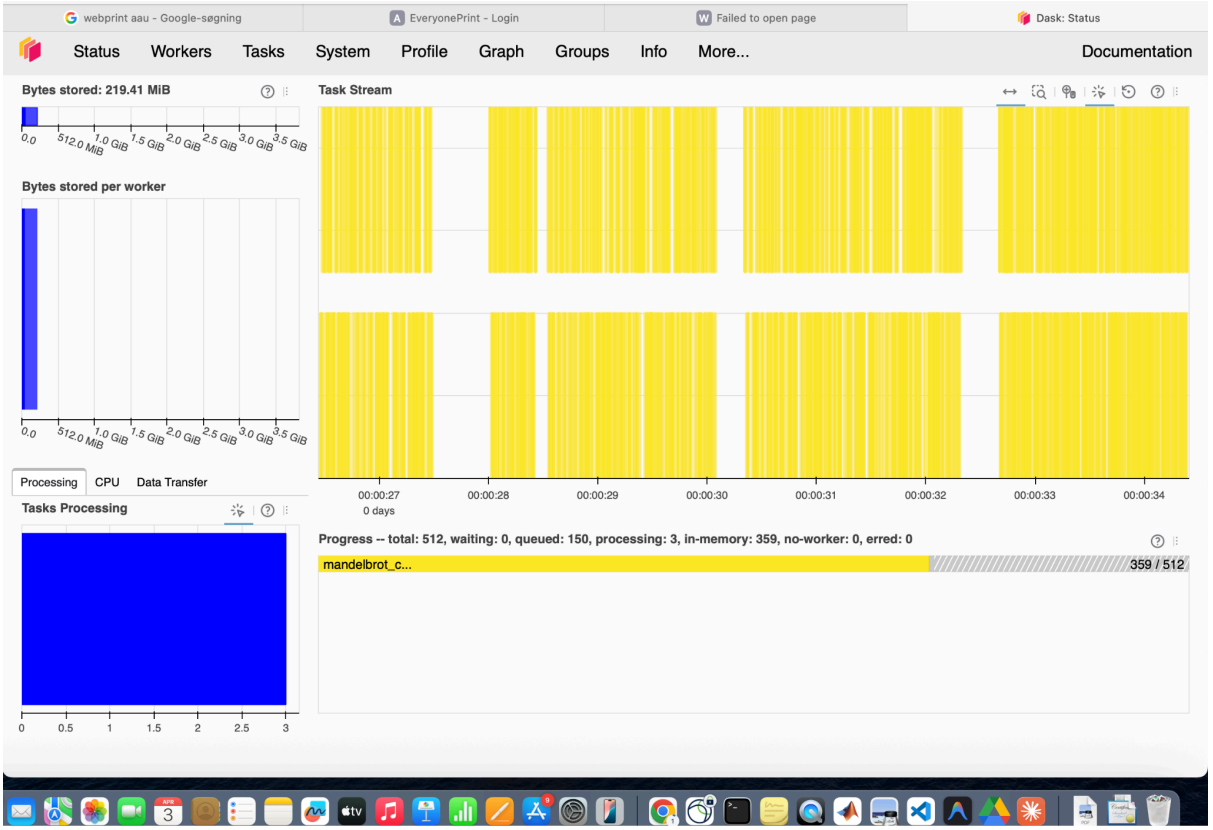


```
Warming up serial and Dask JIT...
Computing serial baseline...
Verifying correctness...
✓ Correctness verified

Serial baseline: 0.0358s

n_chunks |   time (s) |      LIF
-----------------------------------
       1 |     0.0604 |   12.508
       2 |     0.0453 |    9.125
       4 |     0.0408 |    8.123
       8 |     0.0356 |    6.964
      16 |     0.0396 |    7.846
      32 |     0.0470 |    9.513
      64 |     0.0546 |   11.207
-----------------------------------
Optimal n_chunks: 8
Minimum time: 0.0356s
Minimum LIF: 6.964

Plot saved as dask_m2_sweep.png
Dask cluster closed.
```

MP2 M3: Full Benchmark

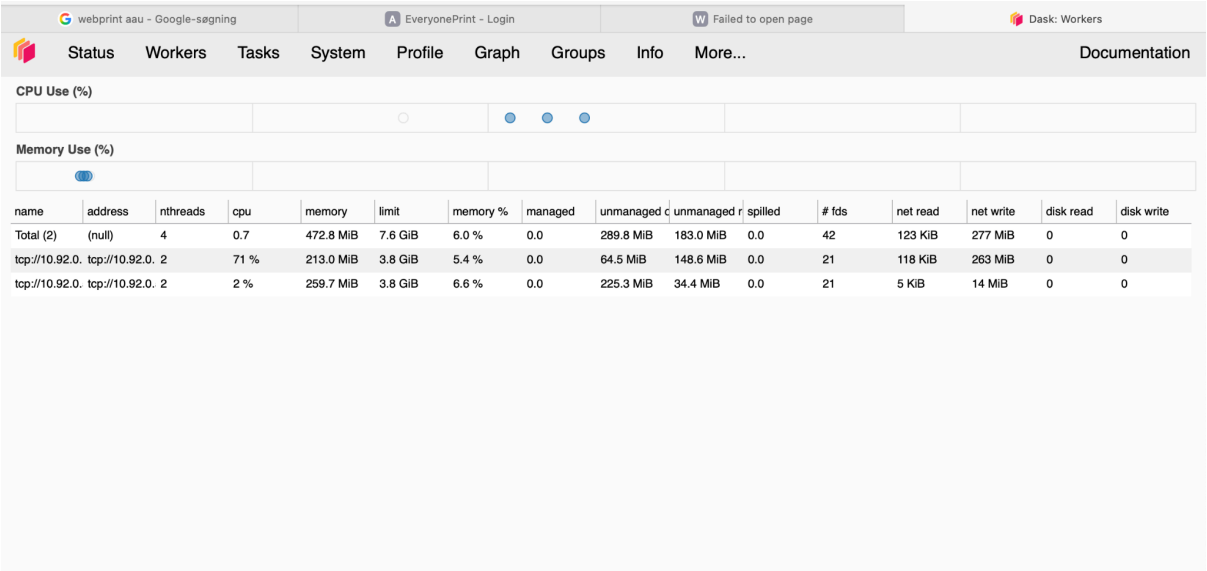
One worker



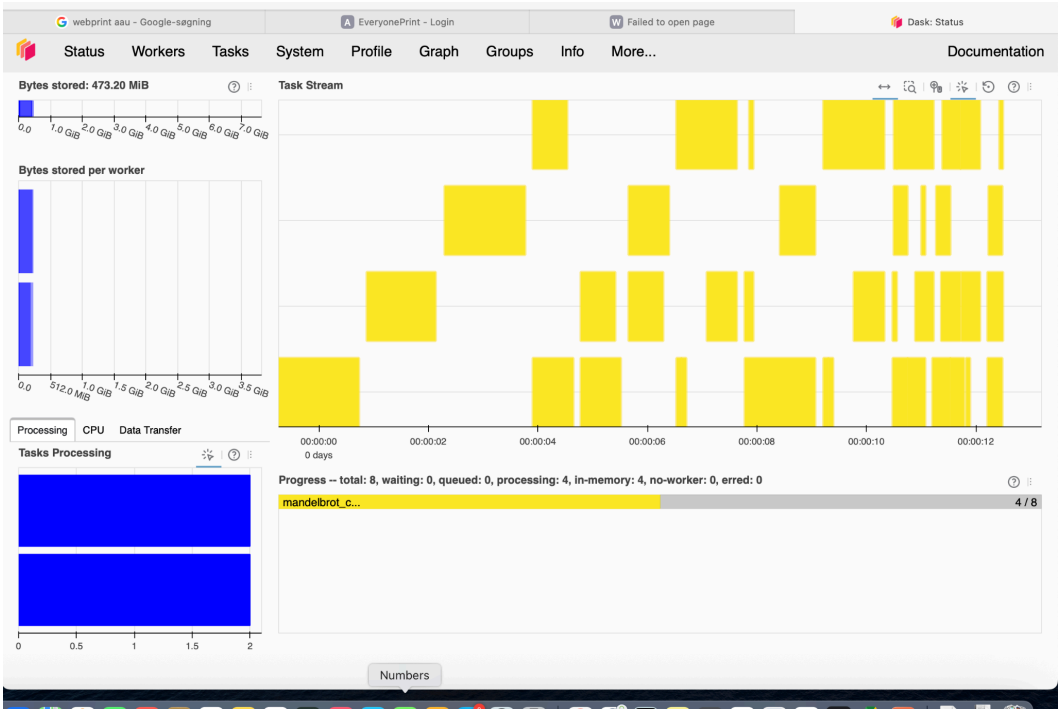


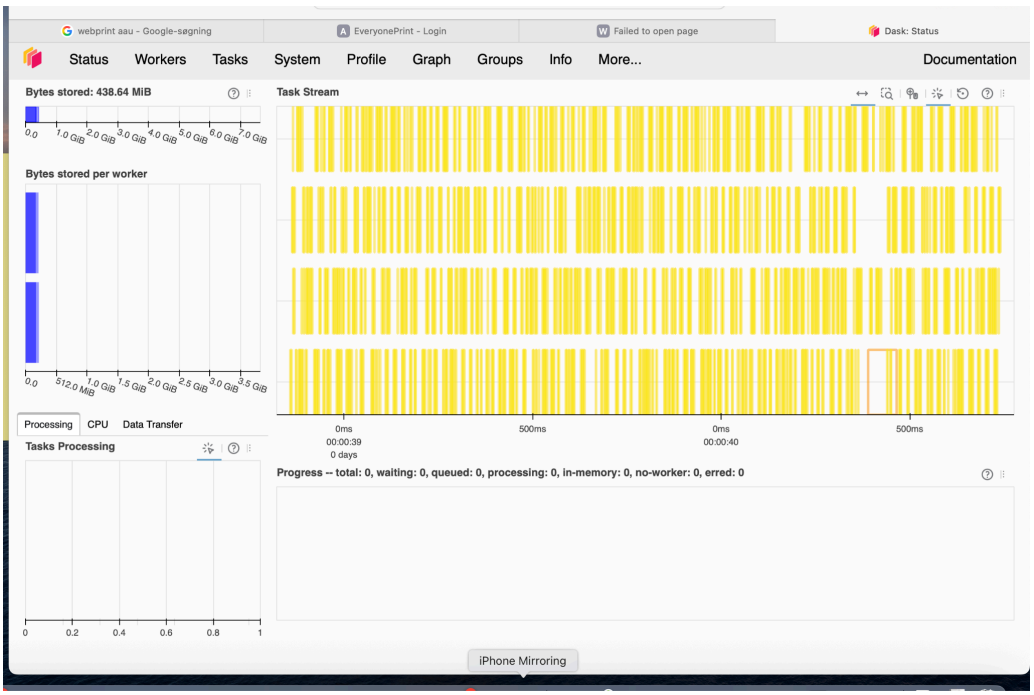

```
(nsc) ubuntu@nsc-head-node-1:~$ python ~/mp2_m1_benchmark.py
============================================================
MP2 M1 BENCHMARK: DISTRIBUTED DASK (STRATO CLUSTER)
============================================================
Grid size N: 4096
Target Scheduler: tcp://10.92.0.225:8786

Computing Serial Numba Baseline (Warm-up)...
Serial Numba time: 1.2621s

Connecting to Dask Cluster at tcp://10.92.0.225:8786...
✓ Connected! Workers detected: 1
Warming up Numba on remote workers...

--- Running Chunk Size Sweep ---
Chunks:    1 | Median Wall Time:  1.6245s
Chunks:    2 | Median Wall Time:  1.6076s
Chunks:    4 | Median Wall Time:  1.6059s
Chunks:    8 | Median Wall Time:  1.6189s
Chunks:   16 | Median Wall Time:  1.6257s
Chunks:   32 | Median Wall Time:  1.6491s
Chunks:   64 | Median Wall Time:  1.7246s
Chunks:  128 | Median Wall Time:  1.9054s
Chunks:  256 | Median Wall Time:  2.2543s
Chunks:  512 | Median Wall Time:  2.8867s
Chunks: 1024 | Median Wall Time:  4.2413s

============================================================
FINAL SUMMARY
============================================================
Numba Serial Baseline: 1.2621s
Best Distributed Time: 1.6059s
Max Speedup Achieved:  0.79x
Optimal n_chunks:      4

✓ Results saved to mp2_m1_exp_distributed.csv
```

2 workers



name	address	nthreads	cpu	memory	limit	memory %	managed	unmanaged c	unmanaged r	spilled	# fds	net read	net write	disk read	disk write
Total (2)	(null)	4	0.7	472.8 MiB	7.6 GiB	6.0 %	0.0	289.8 MiB	183.0 MiB	0.0	42	123 KiB	277 MiB	0	0
tcp://10.92.0.	tcp://10.92.0.	2	71 %	213.0 MiB	3.8 GiB	5.4 %	0.0	64.5 MiB	148.6 MiB	0.0	21	118 KiB	263 MiB	0	0
tcp://10.92.0.	tcp://10.92.0.	2	2 %	259.7 MiB	3.8 GiB	6.6 %	0.0	225.3 MiB	34.4 MiB	0.0	21	5 KiB	14 MiB	0	0






```
(nsc) ubuntu@nsc-head-node-1:~$ python ~/mp2_m1_benchmark.py
============================================================
MP2 M1 BENCHMARK: DISTRIBUTED DASK (STRATO CLUSTER)
============================================================
Grid size N: 4096
Target Scheduler: tcp://10.92.0.225:8786

Computing Serial Numba Baseline (Warm-up)...
Serial Numba time: 1.2587s

Connecting to Dask Cluster at tcp://10.92.0.225:8786...
✓ Connected! Workers detected: 2
Warming up Numba on remote workers...

--- Running Chunk Size Sweep ---
Chunks:    1 | Median Wall Time:  1.6082s
Chunks:    2 | Median Wall Time:  0.8742s
Chunks:    4 | Median Wall Time:  1.2620s
Chunks:    8 | Median Wall Time:  0.8787s
Chunks:   16 | Median Wall Time:  0.9415s
Chunks:   32 | Median Wall Time:  0.8665s
Chunks:   64 | Median Wall Time:  0.9536s
Chunks:  128 | Median Wall Time:  1.0382s
Chunks:  256 | Median Wall Time:  1.3053s
Chunks:  512 | Median Wall Time:  1.7015s
Chunks: 1024 | Median Wall Time:  2.5182s

============================================================
FINAL SUMMARY
============================================================
Numba Serial Baseline: 1.2587s
Best Distributed Time: 0.8665s
Max Speedup Achieved:  1.45x
Optimal n_chunks:      32

✓ Results saved to mp2_m1_exp_distributed.csv
(nsc) ubuntu@nsc-head-node-1:~$
```

Result analysis

For local execution on a single machine, multiprocessing.pool() is often the superior choice over Dask Local because it has significantly lower overhead. Dask is more robust its requires scheduler process that make many small delays. For these mandelbrot calculation , which are relatevly faster , the extra layer of dask (like statemanagement and dashboard) actualy make it slightely lower than lightweight multi processing pool.

Moving to Dask cluster is worth when the work load become too vast for a single computer . even though the cluster is slower for a small grid (may be due to small network bottleneck on strado). It would be the only option if we need to calculate massive grid that would crash laptops RAM. In short , the cluster is for scale while local multi processing is for speed on small tasks.

The surprising result was that my laptop MacBook(0.24) outperforms the strado cluster(0.87s). This is very critical lesson in HPC : raw CPU power is not everything. Laptop shared memory allows zero delayed communication between cores , but strado VM send data through a virtual network which act as a major bottleneck.